

计算机系统基础

一、计算机工作基本原理填空

1、在对某程序进行调试时，看到如下一段信息：

004116DA	83 7D FC 00	cmpl	
004116DE	75 08	jne	004116E8
004116E0	8B 45 FC	mov	
004116E3	89 45 F8	mov	
004116E6	EB 07	jmp	004116EF
004116E8	C7 45 F8 01 00 00 00	movl	
004116EF	E8 2D FB FF FF	callq	func (00411221) ; func 为函数名
004116F4	C7 45 F4 21 12 41 00	movl	\$0x411221, -0x0c(%ebp)
004116FB	FF 55 F4	callq	-0x0c(%ebp)
.....			

① 观察指令在内存中的存放形式(每个空对应一个 16 进制字节数据) (2 分)：

004116DA 83 7D FC 00 75 08 8B 45
004116E2

② 设当前 eip 为 0x004116DE，在取出 eip 指向的指令并进行译码后，eip = 0x004116E0；

③ 在 0x004116DE 处的指令为：75 08 jne 004116E8，直观上是在 zf = 0 时，会将 0x004116E8 → eip，计算出该地址值的方法 0x004116E0(下一条指令的地址) + 08；当转移条件不成立时，则该指令执行完成，eip 不会（会、不会）改变。

④ 004116EF 处的指令中有 0xFFFFFB2D，计算出该值的方法是 00411221 - 004116F4；

⑤ 004116FB 处的子程序调用指令，调用的子程序的入口地址是 0x411221，执行该处语句时，CPU 会将 0x 004116FE 压入堆栈中。

⑥ 执行子程序中的 RET 指令时，CPU 会 从栈顶弹出一个双字 → eip。

2、设一个 C 语言程序中定义有全局数组 int array[5]； array 的起始地址(即 &array[0]) 为 0x420100，

解答内容不得超过装订线

每个元素占 4 个字节，现要将数组的第 2 个元素(即 array[2])置为 20。按指定的寻址方式写出实现该功能的机器指令段（汇编语句段）。

注：完成该功能的 C 语句有：“array[2]=20; *(array +2) =20; int i=2; array[i]=20; int *p=array; p[i] = 20;”等等。不同的 C 语句写法编译后，生成的机器指令不同，会出现多种寻址方式访问同一单元。

- ① 使用直接寻址方式，机器指令中含有操作数的偏移地址：

movl \$20, 0x420108 或 mov dword ptr [420108h], 20

- ② 使用寄存器间接寻址方式，先将操作数的地址送 ebx，再访存：

lea 0x420108, %ebx 或 lea ebx, dword ptr [420108h] ; 给 ebx 赋值

movl \$20, (%ebx) 或 mov dword ptr [ebx], 20

- ③ 使用变址寻址方式，先将元素的下标（即 2）送 eax，再访存：

mov \$2, %eax 或 mov eax, 2 ; 给 eax 赋值

movl \$20, 0x420100(, %eax, 4) 或 mov dword ptr [420100h+eax*4], 20

- ④ 使用基址加变址寻址方式，先将数组的起始地址送 eax，第 2 个元素在数组中的偏移字节数送 ebx，再访存：

lea 0x420100, %eax 或者 lea eax, array ; 给 eax 赋值

mov \$8, %ebx 或者 mov ebx, 8 ; 给 ebx 赋值

movl \$20, (%eax, %ebx, 1) 或者 mov dword ptr [eax+ebx], 20

- ⑤ 对于全局变量，其空间分配在数据段。而对于非静态的局部变量，其空间分配在堆栈段，对应的地址表达方式一般为 disp(%ebp)、disp(%esp) 等。

二、 数据存储及 C 语句转换填空

在Linux环境下，对一个C语言程序进行编译、链接、调试运行，程序片段如下。

```
int fadd(int a, int b)
{
    int temp;

    temp = a + b;

    return temp;
}
```

```

void main( )
{
    int  x = 0x1234;

    int  y = -32;

    int  result = 0;

    char msg[6] = "abc12";    // '1' 的 ASCII 是 0x31, 'a' 的 ASCII 是 0x61

    result = fadd(x, y);

    result = *(int *) (msg+1);
}

```

调试时，设变量 x 的地址（即&x）为 0xffffd508； y 的地址（即&y）为 0xffffd50c，result 的地址为 0xffffd510，数组 msg 的起始地址为 0xffffb516。

- 1、执行到“result=fadd(x,y);”时，以字节为单位观察内存内容（用 16 进制数的形式填空，最左边是内存地址）。

0xffffd508	<u>34</u>	<u>12</u>	<u>00</u>	<u>00</u>	<u>E0</u>	<u>FF</u>	<u>FF</u>	<u>FF</u>
0xffffd510	<u>00</u>	<u>00</u>	<u>00</u>	<u>00</u>	XX	XX	<u>61</u>	<u>62</u>
0xffffd518	<u>63</u>	<u>31</u>	<u>32</u>	<u>00</u>	XX	XX	XX	XX

- 2、数据传送指令解读

“int x = 0x1234;”对应的机器指令为：movl \$0x1234, -0x20(%ebp)，执行该语句时，ebp=0x ffffd528。

“int result = 0;”对应的反汇编指令为 movl \$0, -0x18(%ebp)。

执行“result = *(int *) (msg+1);”后，result 中的值为 0x 32316362。

- 3、函数调用语句解读

语句“result = fadd(x, y);”对应的反汇编代码（最左边的是机器指令的地址）如下。

0x56556210 <+72>:	push -0x1c(%ebp)
0x56556213 <+75>:	push -0x20(%ebp)
0x56556216 <+78>:	call 0x5655619d <fadd>
0x5655621b <+83>:	add \$0x8, %esp
0x5655621e <+86>:	mov %eax, -0x18(%ebp)

	0xffffd4ec
	0xffffd4f0
0x5655621b	0xffffd4f4
0x00001234	0xffffd4f8
0xffffffe0 或 -32	0xffffd4fc
XXXXXXXXXX	0xffffd500

设执行“result = fadd(x, y);”之前，esp 的值为 0xffffd500。

① 在表格的适当位置填写刚进入函数 fadd 内部时，堆栈中存放的相关数据；

② 刚进入函数 fadd 内部时，esp = 0x ffffd4f4；

③ 执行 “add \$0x8, %esp” 之后，esp = 0x ffffd500。

4. 函数 fadd 的指令解读

函数体对应的反汇编代码有：

```
0x5655619d <+0>:    push  %ebp
0x5655619e <+1>:    mov   %esp, %ebp
0x565561a0 <+3>:    sub   $0x10, %esp
0x565561a3 <+6>:    mov   0x8(%ebp), %edx
0x565561a6 <+9>:    mov   0xc(%ebp), %eax
0x565561a9 <+12>:   add   %edx, %eax
0x565561ab <+14>:   mov   %eax, -0x4(%ebp)
0x565561ae <+17>:   mov   -0x4(%ebp), %eax
0x565561b1 <+20>:   .....
```

① 函数参数 a 的地址（即&a）是 0x ffffd4f8。

② 局部变量 temp 的地址（即&temp）是 0x ffffd4ec。

③ 执行 “add %edx, %eax” 后，CF= 1，SF= 0，ZF= 0，OF= 0。

④ 在函数的结束处，有程序段

```
mov %ebp, %esp
pop %ebp
ret
```

执行后可返回到调用函数的断点处。

三、 程序阅读与理解

1、阅读下面的程序，回答问题。

```
void func ()
{
    unsigned short us = 65535;
    unsigned int ui;
    short s = -1;
    int i;
    int x = 0, y = 0;           ; ①
    ui = us;
    i = s;                      ; ②
    if (ui > 0) x = 1;
    if (i > 0) y = 1;           ; ③
}
```

```

    ui = ui >> 1;           ; >>1 右移一个二进制位
    i = i >> 1;             ; ④
    ui = ~ui;               ; 按位取反
    i = -i;                 ; ⑤
}

```

(1) 执行完 ① 处语句后，

us = 0x ffff s = 0x ffff (2个字节的16进制数)

(2) 执行完 ② 处语句后，

ui = 0x 0000ffff i = 0x ffffff (4个字节的16进制数)

ui=us; 对应的机器指令: movzx 或 movz us, %eax 、 mov %eax, ui

i=s; 对应的机器指令: movsx 或 movs s, %eax 、 mov %eax, i

(3) 执行完 ③ 处语句后，

x = 1 y = 0

if (ui>0) x=1; 对应的机器指令: cmp \$0, ui、 jbe lp1、 mov \$1, x、 lp1:...

if (i >0) y=1; 对应的机器指令: cmp \$0, i、 jle lp2、 mov \$1, y、 lp2:...

(4) 执行完 ④ 处语句后，

ui = 0x 00007fff i = 0x ffffff (4个字节的16进制数)

ui = ui >> 1; 对应的机器指令: shr \$1, ui

i = i >> 1; 对应的机器指令: sar \$1, i

(5) 执行完 ⑤ 处语句后，

ui = 0x ffff8000 i = 0x 00000001 (4个字节的16进制数)

ui = ~ui; 对应的机器指令: not ui

i = -i; 对应的机器指令: neg i

2、阅读下面的程序，回答问题。

```

.section .data
    array: .long 10, -20, 30, -40, 50
    length = (. -array)/4      # length 为array中元数的个数, = 5
    format: .ascii "%d\n\0"

.section .text
.global _start
_start:
    mov $0, %eax
    mov $length, %ecx
    lea array, %edi            # ①
lp_1:
    cmpl $0, (%edi)
    jl lp_2                    # ②
    inc %eax
lp_2:

```

```

add $4, %edi
sub $1, %ecx          # ③
jne lp_1
push %eax
push $format
call printf
mov $1, %eax          # 程序正常退出
mov $0, %ebx
int $0x80

```

1) 上述程序的功能是什么？运行后，屏幕上显示的是什么？

统计 array 数组中非负数的个数并显示。 显示 3

2) 若标号 lp_1 写到 ①处语句前，程序运行的结果是什么？为什么？

显示 5。每次循环都将 array 的地址送 edi，每次循环都是判断数组的第一个元素是否为负数。

3) 若将 ② 处的语句改为 “jb lp_2”，程序运行的结果是什么？

显示 5。jb 是无符号数比较转移，任何无符号数都不低于 0。

4) 若漏写了 ③ 处的语句，程序运行会出现什么现象？为什么？

程序运行异常终止。表面上，%edi 在循环中不断加4，加到值为0时，循环终止。但是随着edi的增加，cmpl \$0, (%edi), 访问的内存单元超出程序空间范围，引起异常。

四、 程序优化。

1、举例说明编写 C 程序或者编译器优化时利用相应原则进行优化的做法（包括优化前的方法，优化后的方法）。

① 提高 CPU 中 cache 的命中率

任务：二维数组求累加和

优化前：二重循环，按列序优先访问

优化后：二重循环，按行序优先

② 提高 CPU 中指令流水线的利用率

任务：一维数组求累加和。

优化前：使用循环，逐个元素相加

优化后：循环展开，消除循环，变成一系列加法语句

③ 使用 CPU 中处理速度更快的指令

任务：表达式计算，将一个数 *9

优化前：用乘法运算，乘 9 运算

优化后：左移 3 个二进制位运算（相当于乘 8），再加原来的操作数

④ 使用 CPU 中单指令多数据流指令或串操作指令

任务：两个数组相加，得到一个新数组

优化前：用循环的方法，逐个对应元素相加

优化后：成组运算，一次多对数据同时运算，减少了循环次数

另一个例子：将一个数组中所有元素置 0。优化前，用循环的方法，逐个元素置 0。优化后，用 `memset` 函数实现，该函数的实现封装了串操作指令。

⑤ 使用多核 CPU 中多线程处理能力

任务：两个数组相加，得到一个新数组

优化前：逐个对应元素相加

优化后：创建两个或者多个线程，每个线程完成数组中部分元素（一些行）的求和，即按行将数据分成几块，每个线程完成一个块的相加操作。

五、 链接和异常控制流问答

1、 设一个函数中有语句 `int temp=global;` 其中 `global` 是一个初值为 35 的 `int` 类型全局变量。

编译器对 `global` 的定义(`int global=35;`)和 `temp=global` 编译时，分别会在可重定位目标文件中的哪些节生成哪些信息？

对于 `int global=35;` 在 数据节(.data) 节存放 35 (0x00000023)； 在 字符串节(.strtab)，存放字符串“global”； 在符号表节(.symtab)，存放有关符号 `global` 的信息，包括定义该符号的节号、在相应节(.data)节的地址、数据长度（4 个字节）、属性信息等。

对于 `int temp=global;` 在代码节(.text)中， 生成相应的语句，其中 `global` 的地址（偏移量）用占位符 `0x00000000`。在代码节的重定位节(.rel.text)，要记录 代码节的相应位置（`global` 占位符的起始地址）要被什么符号（符号表的哪一项）、用什么定位方式（地址相对程序计算器 PC 的 32 位偏移）所代替，还包括计算地址时的附加项等。

2、 什么是中断和异常？两者有何差别？什么是中断描述符表？中断和异常的响应过程是什么？

中断和异常：一个进程在执行过程中，正常的逻辑控制流被特殊的事件所打断，CPU 转

到处理这些事件的内核程序去执行，从而引发一个异常控制流。

中断一般是指 CPU 之外的事件发生，如按键盘，鼠标等，是与当前正在执行的指令无关的异步事件。异常是 CPU 正在执行的指令引发的事件，如除 0、访问数据的地址超出程序空间范围等等，是与正在执行的指令相关的同步事件。异常分为故障、陷阱和终止。

中断描述表（中断矢量表）：中断服务程序和异常处理程序的入口地址构成的一个表，每个表项占 8 个字节，存放入口偏移地址及相应代码段的描述信息；这些表项是按照中断和异常的编号（即中断和异常的类型号）顺序排列的。

中断和异常的响应过程：CPU 的控制逻辑确定检测到的中断和异常类型号，从中断描述表取出对应的表项，计算相应的处理程序的入口地址，保存当前的 CS、EIP、EFLAGS 等信息，然后转到处理程序的入口地址对应的位置去执行。在中断处理程序中有 IRET 指令，执行该指令从堆栈中恢复 CS、EIP、EFLAGS 等，从而回到被中断的位置继续执行。对于三类异常：故障、陷阱和终止的处理策略有所差异。故障表示能够修复，引起故障的指令会被再次执行；陷阱则是执行引发陷阱的下一条指令；而终止就要结束程序的运行。